

1. Adam is working in an IT company. He has been given a task to reduce the load of a system by killing some of the processes running in the LINUX operating system. Which commands will he use to complete the given task with the help of the following operation?

- (i) Kill processes by name
- (ii) Kill a process based on the process name
- (iii) Kill a single process at a time with the given process ID

OUTPUT:

```
athar@LAPTOP-8HI3V5MB MSYS ~  
$ taskkill //IM notepad.exe  
SUCCESS: Sent termination signal to the process "Notepad.exe" with PID 12360.  
  
athar@LAPTOP-8HI3V5MB MSYS ~  
$
```

2. Write a program for process creation using C

- (i) Orphan Process

CODE:

```
GNU nano 8.7  
#include <stdio.h>  
#include <unistd.h>  
  
int main() {  
    pid_t pid = fork();  
  
    if (pid > 0) {  
        // Parent process  
        printf("Parent process PID: %d\n", getpid());  
        sleep(2); // Parent exits early  
        printf("Parent exiting...\n");  
    }  
    else if (pid == 0) {  
        // Child process  
        sleep(5); // Child runs longer  
        printf("Child process PID: %d\n", getpid());  
        printf("New Parent PID (init): %d\n", getppid());  
    }  
    else {  
        printf("Fork failed\n");  
    }  
  
    return 0;  
}
```

OUTPUT:

```
athar@LAPTOP-8HI3V5MB MSYS ~  
$ nano hello.c  
  
athar@LAPTOP-8HI3V5MB MSYS ~  
$ gcc hello.c -o hello  
  
athar@LAPTOP-8HI3V5MB MSYS ~  
$ ./hello  
Parent process exiting...  
Parent PID: 1302  
  
athar@LAPTOP-8HI3V5MB MSYS ~  
$ Child process running...  
Child PID: 1303  
New Parent PID (init/systemd): 1
```

(ii) Zombine Process CODE:

```
GNU nano 8.7  
#include <stdio.h>  
#include <unistd.h>  
  
int main() {  
    pid_t pid = fork();  
  
    if (pid > 0) {  
        // Parent process  
        printf("Parent process PID: %d\n", getpid());  
        sleep(10); // Parent wait() call nahi karta  
        printf("Parent exiting...\n");  
    }  
    else if (pid == 0) {  
        // child process  
        printf("Child process PID: %d\n", getpid());  
        printf("Child exiting...\n");  
    }  
    else {  
        printf("Fork failed\n");  
    }  
  
    return 0;  
}
```

OUTPUT:

```
athar@LAPTOP-8HI3V5MB MSYS ~  
$ gcc hello.c -o hello  
  
athar@LAPTOP-8HI3V5MB MSYS ~  
$ ./hello  
Parent exiting  
  
athar@LAPTOP-8HI3V5MB MSYS ~  
$ Child Process  
PID : 1193  
PPID: 1
```

3. Create the process using fork () system call.

- (i) Child Process creation
- (ii) Parent process creation
- (iii) PPID and PID

CODE:

```
GNU nano 8.7 hello.c  
#include <stdio.h>  
#include <unistd.h>  
  
int main() {  
    pid_t pid;  
  
    pid = fork(); // create child process  
  
    if (pid < 0) {  
        printf("Fork failed\n");  
    }  
    else if (pid == 0) {  
        // Child process  
        printf("Child Process Created\n");  
        printf("Child PID : %d\n", getpid());  
        printf("Parent PID : %d\n", getppid());  
    }  
    else {  
        // Parent process  
        printf("Parent Process Created\n");  
        printf("Parent PID : %d\n", getpid());  
        printf("Child PID : %d\n", pid);  
    }  
  
    return 0;  
}
```

OUTPUT:

```
athar@LAPTOP-8HI3V5MB MSYS ~  
$ ./hello  
Parent Process  
PID: 1200  
Child PID: 1201  
Child Process  
PID: 1201  
PPID: 1200
```